

Guide d'Implémentation Pas-à-Pas : Backend Django REST

Exemples de Code & Configurations pour : Filtrage, Webhooks Paiement, Celery & Signals Notifications

Étape 1 : Filtrage des Concours selon les Préférences Utilisateur

Pour permettre à un étudiant de filtrer les concours selon son domaine, nous combinons `django-filter` et l'optimisation ORM pour éviter les requêtes inutiles.

```
# filters.py
import django_filters
from .models import Concours

class ConcoursFilter(django_filters.FilterSet):
    min_date = django_filters.DateFilter(field_name="date_limite", lookup_expr='gte')
    filiere = django_filters.CharFilter(field_name="filiere_nom", lookup_expr='icontains')

    class Meta:
        model = Concours
        fields = ['statut', 'filiere', 'min_date']

# views.py
from rest_framework import viewsets
from django_filters.rest_framework import DjangoFilterBackend

class ConcoursViewSet(viewsets.ReadOnlyModelViewSet):
    # select_related & prefetch_related évitent le problème de $N+1$ requêtes SQL
    queryset = Concours.objects.select_related('filiere').prefetch_related('sessions').all()
    serializer_class = ConcoursSerializer
    filter_backends = [DjangoFilterBackend]
    filterset_class = ConcoursFilter
```

Étape 2 : Traitement des Paiements Mobile Money (Webhooks & Atomacité)

Lorsqu'Orange Money ou MTN valident un paiement, ils envoient un Webhook `POST`. La transaction doit être atomique (succès total ou annulation).

```
# views.py
from django.db import transaction
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import AllowAny
from rest_framework.response import Response
from .models import Transaction, AccesConcours

@api_view(['POST'])
@permission_classes([AllowAny]) # Le webhook vient du serveur de paiement
def mobile_money_webhook(request):
    data = request.data
    ref_externe = data.get("transaction_id")
    statut = data.get("status")

    # Isolation de la transaction bancaire en base de données
    with transaction.atomic():
        tx = Transaction.objects.select_for_update().get(reference=ref_externe)
        if tx.statut == 'SUCCESS':
            return Response({"message": "Déjà traité"}, status=200)

        if statut == 'SUCCESS':
            tx.statut = 'SUCCESS'
            tx.save()
            # Déblocage de l'accès à l'épreuve/concours pour l'utilisateur
            AccesConcours.objects.create(user=tx.user, concours=tx.concours)

    return Response({"status": "ok"})
```

Étape 3 : Tâches Asynchrones & Planifiées (Celery + Redis)

Pour mettre à jour automatiquement l'état d'un concours dont la date est expirée sans bloquer l'API, nous utilisons Celery Beat.

```
# tasks.py
from celery import shared_task
from django.utils import timezone
from .models import Concours

@shared_task
def verifier_et_cloturer_concours():
    maintenant = timezone.now()
    # Recherche et clôture automatique en arrière-plan
    concours_expires = Concours.objects.filter(date_limite__lt=maintenant, statut='OUVERT')
    nb_mis_a_jour = concours_expires.update(statut='CLOTURE')
    return f"{nb_mis_a_jour} concours clôturés automatiquement."
```

Étape 4 : Déclencheur Automatique de Notifications (Django Signals)

Lorsqu'un administrateur annule ou modifie un concours, un signal écoute l'enregistrement et déclenche l'envoi de notifications aux candidats.

```
# signals.py
from django.db.models.signals import post_save
from django.dispatch import receiver
from .models import Concours, Notification
from .tasks import envoyer_notification_push_async

@receiver(post_save, sender=Concours)
def notifier_changement_concours(sender, instance, created, **kwargs):
    if not created and instance.statut == 'ANNULE':
        # Récupération des abonnés au concours
        candidats = instance.candidats.all()
        for candidat in candidats:
            Notification.objects.create(
                user=candidat,
                titre=f"Concours Annulé : {instance.nom}",
                message="Le concours auquel vous êtes inscrit a été annulé."
            )
        # Appel asynchrone via Celery vers Firebase / Service SMS
        envoyer_notification_push_async.delay(candidat.id, instance.nom)
```